

Similarity-Based Learning

What is Similarity-Based Learning

Similarity-based learning refers to methods that make predictions by comparing a new case to previously observed cases and transferring information from the most similar examples. Rather than fitting a global parametric model, these approaches rely on a distance metric (e.g., Euclidean, Manhattan) or a similarity function (e.g., cosine) to define neighbourhoods in feature space; predictions are then derived from those neighbourhoods (Müller & Guido, 2016; Kelleher, Mac Namee, & D’Arcy, 2020). The core intuition is local inference: examples that are “close” under a suitable metric are expected to have similar outputs. Because the “model” is primarily the representation plus the metric, careful feature engineering and scaling are central to performance (Albon, 2018; Müller & Guido, 2016).

Key algorithms and measures

k-Nearest Neighbours (k-NN)

k-NN is the canonical similarity-based method. Training is trivial: store the labelled dataset. At prediction time, locate the k closest training points to the query (under a chosen metric) and aggregate their targets: majority vote for classification or the mean for regression (Müller & Guido, 2016). The hyperparameter k controls the bias–variance trade-off: small k yields highly flexible, potentially overfitting decision boundaries; larger k smooths decisions but may underfit. Weighted versions or distance weighting improve robustness when neighbour densities vary (Kelleher et al., 2020).

Distance weighting: When combining the neighbours’ labels or values, give closer neighbours a bigger influence than farther ones (intuitive: the closest cases should “speak” louder).

Cosine similarity

Cosine similarity measures the angle between two vectors: the dot product of x and y divided by the product of their Euclidean norms. It is scale-invariant, emphasizing orientation rather than magnitude. After L2 normalization, two documents that point in the same direction have a cosine similarity of 1 regardless of length; this is why cosine is standard for sparse, high-dimensional text representations such as TF-IDF (Müller & Guido, 2016).

Treat two examples as similar if they point in the same direction after rescaling; we care about pattern, not overall size.

Euclidean (L2) and Manhattan (L1) distances

Euclidean distance is the square root of the sum of squared coordinate differences; Manhattan distance is the sum of absolute differences. Both are exceptional cases of the Minkowski family (with $p = 2$ and $p = 1$, respectively) and are widely available in software libraries (Albon, 2018). Because distance magnitudes are sensitive to feature scale, standardization (zero-mean/unit-variance) or min–max scaling is often essential to prevent a single large-range feature from dominating the metric (Albon, 2018; Müller & Guido, 2016). L1 distances are typically more robust to outliers than L2 because they grow linearly with deviations rather than quadratically (Kelleher et al., 2020).

Applications

Recommendation systems

Neighbourhood methods implement intuitive collaborative filtering. In user-based filtering, we compute similarities among user profiles and recommend items favoured by the nearest users; in item-based filtering, we suggest items most similar to those a user already likes. Cosine similarity on normalized interaction vectors is a practical default, and k-NN variants remain strong baselines and useful candidate generators in larger recommender stacks (Kelleher et al., 2020; Müller & Guido, 2016).

Document classification and retrieval

Representing documents with bag-of-words or TF-IDF and applying cosine similarity provides an effective basis for topic relatedness. A k-NN classifier labels a new document by voting among its nearest neighbours; retrieval systems rank by cosine similarity to the query vector. This pipeline (tokenization, stop-word handling, TF-IDF, L2 normalization, neighbour search) is simple to implement and competitive on many baseline text tasks (Müller & Guido, 2016).

Anomaly (outlier) detection

Nearest-neighbour ideas readily support unsupervised outlier detection: an instance is anomalous if its distance to the k-th neighbour is large or if few points fall within a fixed radius. These methods perform well when “normal” behaviour forms dense clusters and anomalies lie in sparse regions (Kelleher et al., 2020; Albon, 2018).

Similarity search for images and signals

In computer vision and audio, engineered or learned embeddings enable fast nearest-neighbour search for look-alike images, near-duplicate detection, or clip matching. The mechanics mirror k-NN: define a representation, choose a metric, and retrieve the nearest examples (Müller & Guido, 2016).

Strengths

- **Simplicity and transparency:** The decision rule (“ask your neighbours”) is easy to explain and audit; showing the actual neighbours offers concrete, example-based justifications (Müller & Guido, 2016).
- **Nonparametric flexibility:** k-NN can model highly nonlinear boundaries without specifying a global functional form (Kelleher et al., 2020).
- **Broad applicability:** With a good representation and metric, the same approach works across tabular data, text, images, and time series, for both classification and regression (Müller & Guido, 2016).
- **Immediate updates:** New labelled examples can be incorporated without retraining by simply adding them to the reference set or index (Kelleher et al., 2020).

Limitations & Mitigations

Limitation	Mitigation
Prediction-time cost (comparing to many stored examples can be slow).	Use efficient neighbour indexes in low/moderate dimensions (k-d trees, ball trees) and approximate nearest-neighbour (ANN) search or dimensionality reduction at higher dimensions (Albon, 2018).
High dimensionality (distances become less informative).	Feature selection, dimensionality reduction, or learned embeddings; remove irrelevant or redundant features (Kelleher et al., 2020; Müller & Guido, 2016).
Sensitivity to scaling and representation (large-range features dominate).	Standardize or normalize features; engineer domain-relevant features (Albon, 2018; Müller & Guido, 2016).
Choosing k and class imbalance (small k overfits; large k oversmooths; majority class dominates).	Tune k with validation, use distance weighting, and apply class-balanced splits and metrics (Kelleher et al., 2020).
Storage footprint (model grows with dataset).	Prototype selection/condensation; compressed indexes; prune outdated or redundant examples (Albon, 2018).

References

Albon, C. (2018). *Machine Learning with Python Cookbook: Practical Solutions from Preprocessing to Deep Learning* (1st ed.). O'Reilly Media, Inc.

Kelleher, J. D., Mac Namee, B., & D'Arcy, A. (2020). *Fundamentals of machine learning for predictive data analytics: Algorithms, worked examples, and case studies* (2nd ed.). MIT Press.

Müller, A. C., & Guido, S. (2016). *Introduction to machine learning with Python: A guide for data scientists*. O'Reilly Media Inc.